

 INSTITUTO SUPERIOR TECNOLÓGICO DE TURISMO Y PATRIMONIO YAVIRAC <i>¡Fortaleciendo Capacidades!</i>		
Grupo: Franco Catagña, Michael de la Torre	Periodo Académico: 2026-01	Carrera: Desarrollo De Software
Asignatura: Devops	Fecha: 04/06/2026	Código:

INVESTIGACIÓN DE GIT

1. Control de versiones

El control de versiones es un sistema que permite registrar y gestionar los cambios realizados en archivos a lo largo del tiempo, facilitando la colaboración entre varios desarrolladores y la recuperación de versiones anteriores.

Utilidad

- Mantener un historial de cambios.
- Recuperar versiones anteriores.
- Facilitar el trabajo colaborativo.
- Evitar la pérdida de información.

Ejemplo práctico

Un equipo desarrolla una aplicación web. Cada cambio realizado queda registrado, permitiendo volver a una versión estable si surge un error.

2. Git

Definición

Git es un sistema de control de versiones distribuido creado por Linus Torvalds en 2005 para gestionar proyectos de software de forma eficiente.

Utilidad

- Registrar cambios en el código.
- Trabajar de manera distribuida.
- Facilitar la colaboración entre desarrolladores.

Ejemplo práctico

Un desarrollador modifica un archivo y utiliza el comando:

```

</> Bash

git add .
git commit -m "Corrección de errores"

```

3. GitHub

Definición

GitHub es una plataforma basada en la nube que permite alojar repositorios Git y colaborar en proyectos de software.

Utilidad

- Almacenamiento remoto de código.
- Gestión de proyectos colaborativos.
- Automatización mediante GitHub Actions.

Ejemplo práctico

Un equipo comparte un repositorio en GitHub para desarrollar una aplicación y gestionar incidencias.

4. Repositorio local

Definición

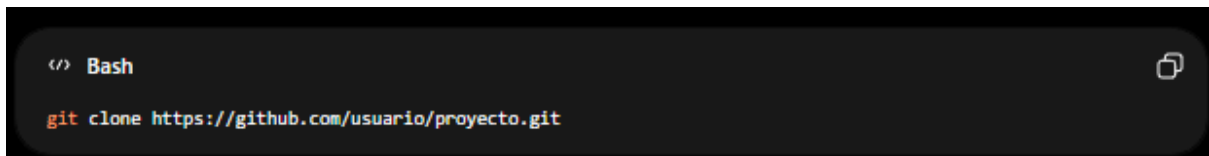
Es la copia del repositorio almacenada en el equipo del desarrollador.

Utilidad

- Permite trabajar sin conexión.
- Almacena el historial completo del proyecto.

Ejemplo práctico

Al ejecutar:

A screenshot of a terminal window with a dark background. The title bar at the top says "Bash" with a code icon on the left and a copy icon on the right. The terminal shows the command `git clone https://github.com/usuario/proyecto.git` in a light blue monospace font.

se crea una copia local del repositorio.

5. Repositorio remoto

Definición

Es la versión del repositorio alojada en un servidor externo, como GitHub.

Utilidad

- Compartir código.
- Sincronizar cambios entre miembros del equipo.

Ejemplo práctico

Un desarrollador envía sus cambios mediante:

A screenshot of a terminal window with a dark background. The title bar at the top says "Bash" with a code icon on the left and a copy icon on the right. The terminal shows the command `git push origin main` in a light blue monospace font.

6. Commit

Definición

Un commit es un registro permanente de los cambios realizados en el repositorio.

Utilidad

- Mantener un historial organizado.
- Documentar avances del proyecto.

Ejemplo práctico

```
</> Bash
git commit -m "Agregar módulo de autenticación"
```

7. Branch (Rama)

Definición

Una rama es una línea independiente de desarrollo dentro de un repositorio.

Utilidad

- Desarrollar funcionalidades sin afectar la versión principal.
- Facilitar el trabajo simultáneo.

Ejemplo práctico

```
</> Bash
git checkout -b nueva-funcionalidad
```

8. Merge

Definición

Merge es el proceso de combinar los cambios de una rama con otra.

Utilidad

- Integrar nuevas funcionalidades al proyecto principal.

Ejemplo práctico

```
</> Bash
git checkout main
git merge nueva-funcionalidad
```

9. Pull Request

Definición

Un Pull Request (PR) es una solicitud para revisar e integrar cambios de una rama en otra dentro de GitHub.

Utilidad

- Revisar código antes de integrarlo.
- Mejorar la calidad del software.

Ejemplo práctico

Un desarrollador crea un PR para que el equipo revise una nueva funcionalidad antes de fusionarla con la rama principal.

10. Fork

Definición

Un fork es una copia personal de un repositorio alojado en GitHub.

Utilidad

- Contribuir a proyectos sin tener permisos directos.
- Experimentar sin afectar el repositorio original.

Ejemplo práctico

Un estudiante realiza un fork de un proyecto de código abierto, implementa mejoras y posteriormente envía un Pull Request.

11. Conflictos de integración

Definición

Ocurren cuando Git no puede determinar automáticamente cómo combinar cambios realizados sobre las mismas líneas de un archivo.

Utilidad

Su resolución garantiza que el código integrado sea coherente.

Ejemplo práctico

Dos desarrolladores modifican la misma línea de un archivo. Al realizar un merge, Git solicita resolver el conflicto manualmente.

12. Integración Continua (CI)

Definición

La Integración Continua es una práctica de desarrollo que consiste en integrar cambios de código frecuentemente y ejecutar pruebas automáticas para detectar errores tempranamente.

Utilidad

- Detectar errores rápidamente.
- Mejorar la calidad del software.
- Reducir problemas de integración.

Ejemplo práctico

Cada vez que un desarrollador realiza un push, se ejecutan pruebas automáticas que verifican el correcto funcionamiento del proyecto.

13. Entrega Continua (Continuous Delivery)

Definición

Es una práctica que garantiza que el software esté siempre listo para ser desplegado en producción mediante procesos automatizados.

Utilidad

- Reducir riesgos en las liberaciones.
- Acelerar la entrega de nuevas funcionalidades.

Ejemplo práctico

Después de aprobar todas las pruebas, la aplicación queda preparada para ser desplegada con una acción manual.

14. Despliegue Continuo (Continuous Deployment)

Definición

Es una extensión de la entrega continua donde cada cambio validado se despliega automáticamente en producción.

Utilidad

- Entrega rápida de funcionalidades.
- Automatización completa del proceso de liberación.

Ejemplo práctico

Cada vez que el código supera las pruebas automáticas, se publica automáticamente en el servidor de producción.

15. GitHub Actions

Definición

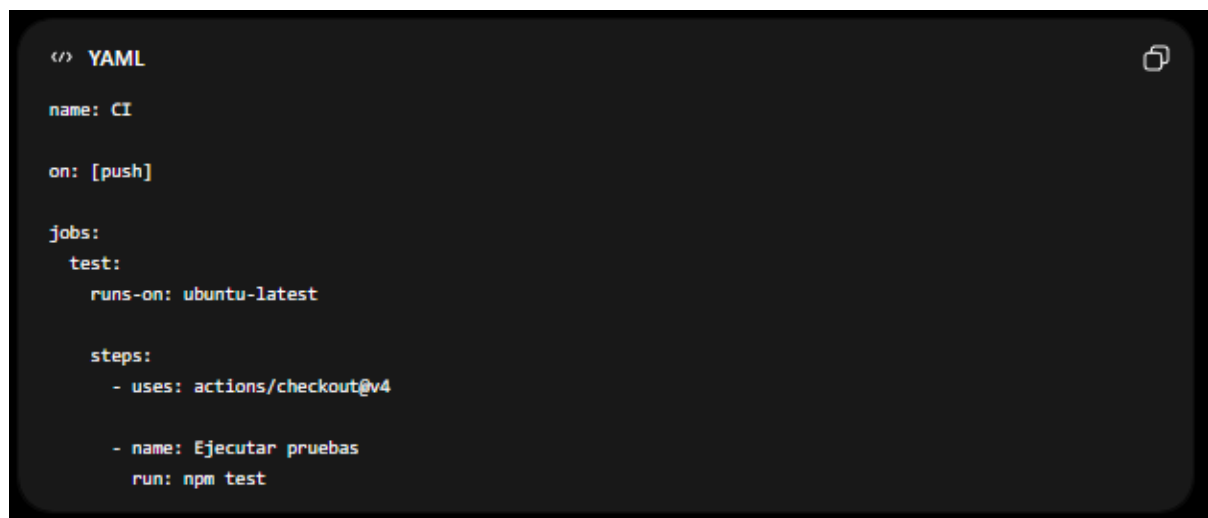
GitHub Actions es un servicio de automatización integrado en GitHub que permite crear flujos de trabajo para compilar, probar y desplegar aplicaciones.

Utilidad

- Automatizar pruebas.
- Ejecutar procesos CI/CD.
- Reducir tareas manuales.

Ejemplo práctico

En un archivo YML:

A screenshot of a code editor showing a GitHub Actions workflow file. The file is named 'YAML' and contains the following content:

```
name: CI

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Ejecutar pruebas
        run: npm test
```

Fuentes bibliográficas:

- <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>
- <https://github.com/>
- <https://git-scm.com/docs/git-clone>
- <https://git-scm.com/book/en/v2/Git-Basics-Working-with-Remotes>
- <https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository>
- <https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell>
- <https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging>
- <https://docs.github.com/en/pull-requests>
- <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/fork-a-repo>
- https://git-scm.com/docs/git-merge#_how_conflicts_are_presented
- <https://martinfowler.com/articles/continuousIntegration.html>
- <https://docs.github.com/en/actions>

EJERCICIO PRÁCTICO

La actividad deberá realizarse exclusivamente en grupos de dos estudiantes. Cada pareja deberá organizar los siguientes roles:

Estudiante A (Michael de la Torre)

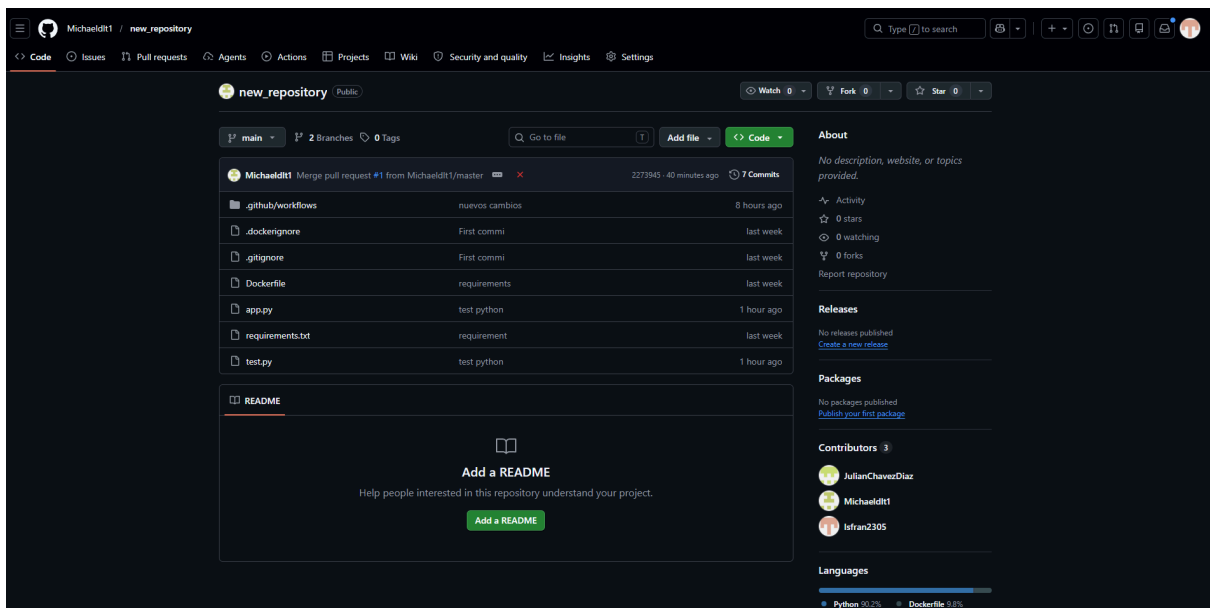
- Creador del repositorio.
- Responsable de revisar Pull Requests.
- Responsable de aprobar cambios.
- Responsable de realizar los Merge.

Estudiante B (Franco Catagña)

- Desarrollador colaborador.
- Responsable de crear ramas.
- Responsable de generar Pull Requests.
- Responsable de integrar nuevas funcionalidades.

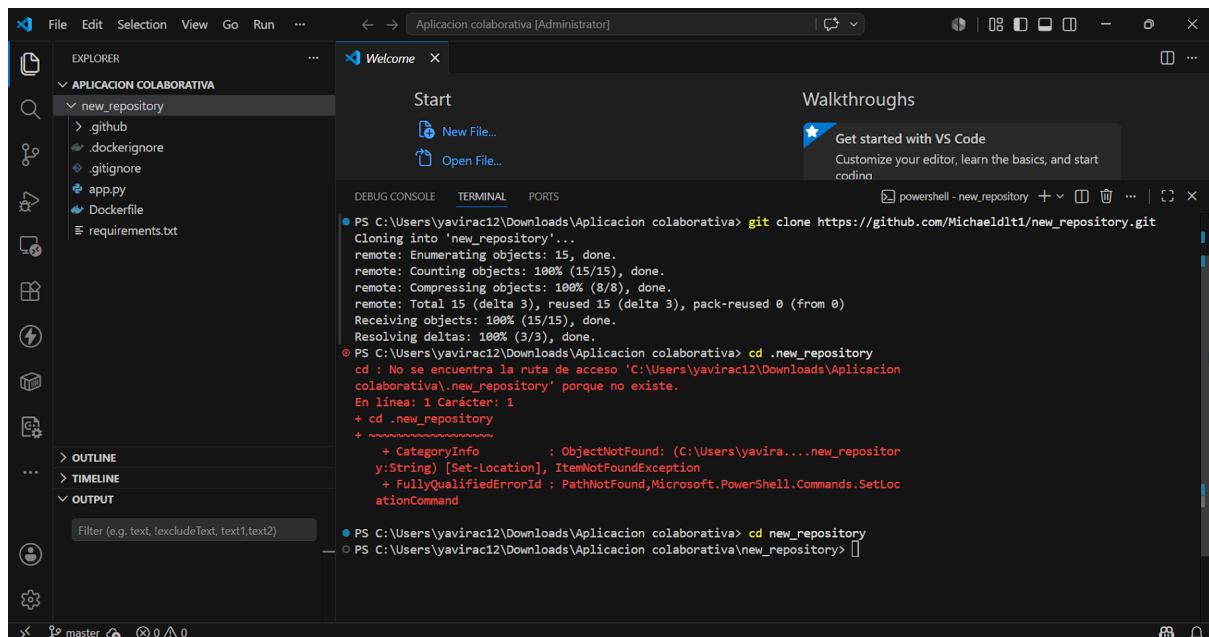
PRUEBAS DE LA PRÁCTICA:

1. Creación del repositorio:



Michael creó un repositorio en GitHub denominado Proyecto-Colaborativo, configurando la rama principal (`main`) después de crear el repositorio inmediatamente invita como colaborar a Franco con el usuario Isfran2305

2. Clonación del repositorio:



The screenshot shows the Visual Studio Code interface with the Explorer panel on the left displaying the file structure of a project named 'APLICACION COLABORATIVA'. The file structure includes a 'new_repository' folder containing '.github', '.dockerignore', '.gitignore', 'app.py', 'Dockerfile', and 'requirements.txt'. The main editor area shows the 'Welcome' page with 'Start' and 'Walkthroughs' sections. The 'Terminal' panel at the bottom displays the following commands and output:

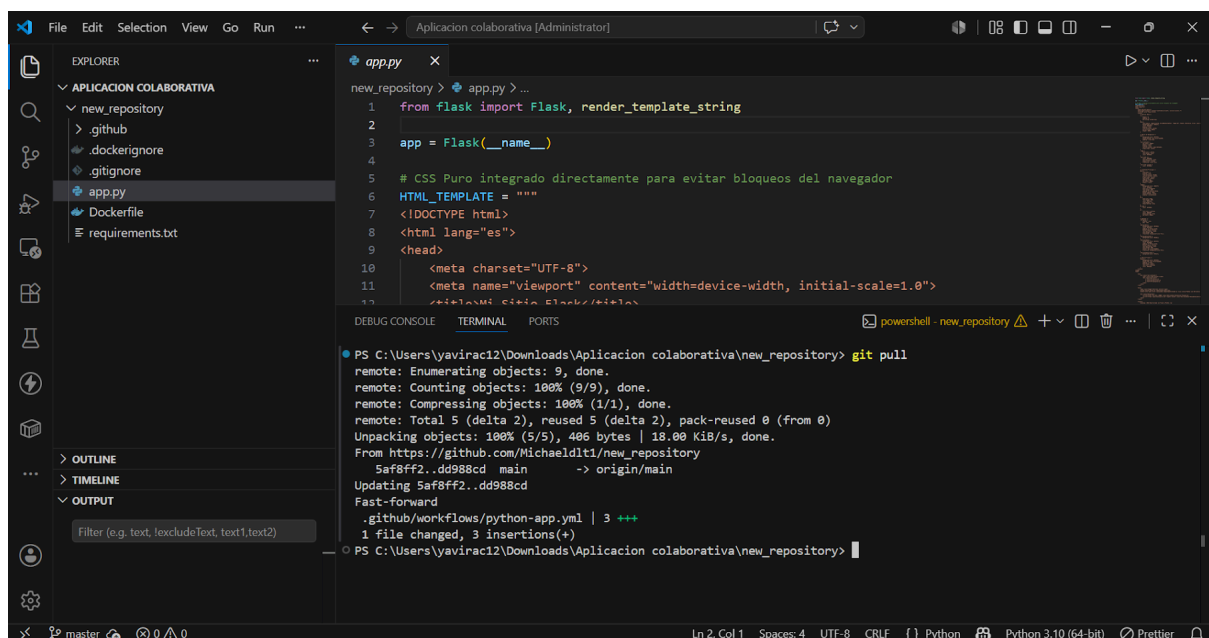
```
PS C:\Users\yavirac12\Downloads\Aplicacion colaborativa> git clone https://github.com/MichaelDlt1/new_repository.git
Cloning into 'new_repository'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 15 (delta 3), reused 15 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (15/15), done.
Resolving deltas: 100% (3/3), done.
PS C:\Users\yavirac12\Downloads\Aplicacion colaborativa> cd .new_repository
cd : No se encuentra la ruta de acceso 'C:\Users\yavirac12\Downloads\Aplicacion colaborativa\.new_repository' porque no existe.
En línea: 1 Carácter: 1
+ cd .new_repository
+ ~~~~~
    + CategoryInfo          : ObjectNotFound: (C:\Users\yavira...new_repositor
    y:String) [Set-Location], ItemNotFoundException
    + FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLoc
    ationCommand
PS C:\Users\yavirac12\Downloads\Aplicacion colaborativa> cd new_repository
PS C:\Users\yavirac12\Downloads\Aplicacion colaborativa\new_repository>
```

Franco aceptó la invitación y procedió a clonar el repositorio en su equipo local a través de “git clone”

Una vez clonado, verificó el contenido inicial y sincronizó la rama principal.

Después Michael hace un cambio leve en el repositorio.

3. Actualización de cambios y creación de una nueva rama:



The screenshot shows the Visual Studio Code interface with the Explorer panel on the left displaying the file structure of a project named 'APLICACION COLABORATIVA'. The file structure includes a 'new_repository' folder containing '.github', '.dockerignore', '.gitignore', 'app.py', 'Dockerfile', and 'requirements.txt'. The main editor area shows the 'app.py' file with the following code:

```
1 from flask import Flask, render_template_string
2
3 app = Flask(__name__)
4
5 # CSS Puro integrado directamente para evitar bloqueos del navegador
6 HTML_TEMPLATE = """
7 <!DOCTYPE html>
8 <html lang="es">
9 <head>
10 <meta charset="UTF-8">
11 <meta name="viewport" content="width=device-width, initial-scale=1.0">
12 <title> Sitio Flask</title>
```

The 'Terminal' panel at the bottom displays the following commands and output:

```
PS C:\Users\yavirac12\Downloads\Aplicacion colaborativa\new_repository> git pull
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (1/1), done.
remote: Total 5 (delta 2), reused 5 (delta 2), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 406 bytes | 18.00 KiB/s, done.
From https://github.com/MichaelDlt1/new_repository
 5af8ff2..dd988cd main    -> origin/main
Updating 5af8ff2..dd988cd
Fast-forward
 .github/workflows/python-app.yml | 3 +++
 1 file changed, 3 insertions(+)
PS C:\Users\yavirac12\Downloads\Aplicacion colaborativa\new_repository>
```

Franco hace el pull para confirmar los cambios.

Una vez terminado de descargar los cambios termina por crear una nueva rama en github.com y la termina llamando “master”.

Search branches...					
Default					
Branch	Updated	Check status	Behind	Ahead	Pull request
main	6 minutes ago	0/1		Default	
Your branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
master	now	0/1	0	0	
Active branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
master	now	0/1	0	0	

4.Trabajo en la rama “master”:

```

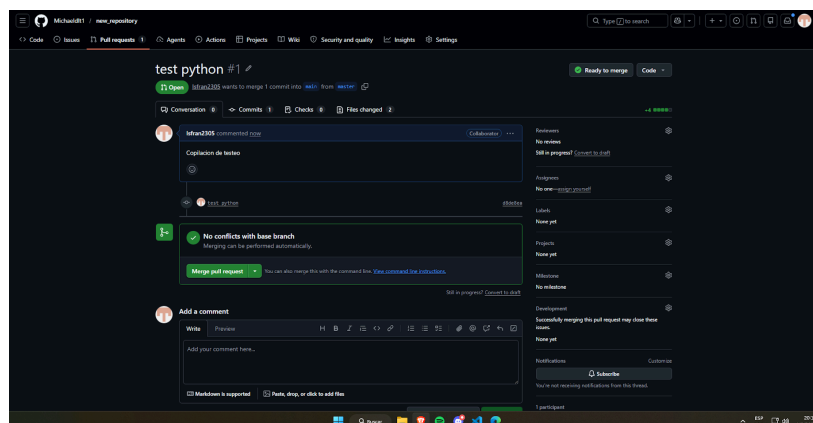
1 def sumar(a, b):
2     return a + b
3
4 # Ejemplo de uso:
5 print(sumar(5, 3)) # Resultado: 8
6
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Usuario\Desktop\Devops\new_repository> git pull origin master
* branch
  master      -> FETCH_HEAD
Already up-to-date.
PS C:\Users\Usuario\Desktop\Devops\new_repository> git push origin master
Everything up-to-date
PS C:\Users\Usuario\Desktop\Devops\new_repository> git add .
PS C:\Users\Usuario\Desktop\Devops\new_repository> git commit -m "test python"
[master d8b8cd] test python
2 files changed, 4 insertions(+)
create mode 100644 test.py
PS C:\Users\Usuario\Desktop\Devops\new_repository> git push origin master
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 389 bytes | 389.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
to https://github.com/Michael111/new_repository.git
d8b8cd..d8b8cd master -> master
PS C:\Users\Usuario\Desktop\Devops\new_repository>

```

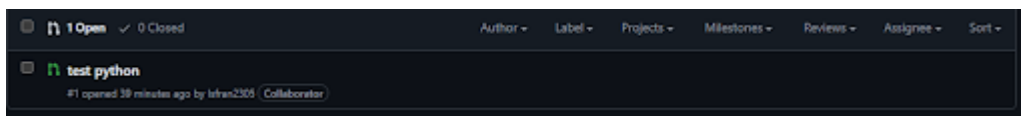
Franco crea un nuevo archivo de python llamado test.py para realizar pruebas, registra los cambios con commit e inmediatamente procede a generar un pull request y proceder a ejecutar un pull request en la página del repositorio.

Michael procede a revisar el código antes de que se efectúen los cambios en el repositorio en la rama principal llamada “main”.

Franco procede a enviar el pull request para ejecutar el merge en el repositorio.



Michael comienza a verificar la solicitud del pull request para poder aceptarlo y poder fusionar ambas ramas.



Tras verificar las correcciones, Michael aprobó el Pull Request y realizó el Merge hacia la rama principal (main).

De esta manera, la nueva funcionalidad quedó integrada oficialmente en el proyecto.

